Smart School Management System

Graduation Project Technical Report

Document generated: June 04, 2026

Full-stack implementation: Django REST API, React SPA, Face Recognition microservice, AI Chatbot, EN/AR localization

Abstract

This report presents the Smart School Management System, a bilingual web platform designed to streamline school operations for administrators, teachers, students, and parents. The system integrates traditional student information management with innovative subsystems: instructor-controlled face-recognition attendance, an AI chatbot backed by Google Gemini with database-scoped context, and English/Arabic localization at the API, database, and microservice layers.

The backend is implemented in Django 5.2 with Django REST Framework and JSON Web Token authentication, persisting data to Microsoft SQL Server. A React 19 single-page application provides role-specific dashboards. A FastAPI microservice performs face detection and matching using 128-dimensional encodings stored on the filesystem. Real-time notifications use Django Channels over WebSocket. Weekly analytics reports aggregate attendance and academic metrics with optional PDF export.

Requirements, architecture, database design, and API contracts are derived from a comprehensive source code audit comprising twelve Django applications, nineteen data models, seventeen REST viewsets, and approximately twenty-seven custom API actions. Testing includes fifty-one localization unit tests and an end-to-end attendance integration script. Results confirm functional achievement of project objectives with documented paths for production hardening and user acceptance evaluation.

Keywords: school management, face recognition, attendance automation, Django, React, bilingual software, chatbot, JWT, microservices.

Table of Contents

1. Introduction

Educational institutions worldwide rely on accurate attendance records, timely communication with families, and transparent academic reporting. Traditional paper registers and disconnected spreadsheets fail to provide real-time visibility for administrators, strain teachers during roll call, and delay parental awareness of absences or underperformance. The proliferation of mobile devices and affordable cameras further enables new modalities for identity verification, while large language models offer natural-language interfaces to institutional data that were previously accessible only through formal reports.

The Smart School Management System addresses these challenges through an integrated web platform that unifies user management, academic structure, face-recognition attendance, multiple-choice examinations, educational content delivery, weekly analytics, in-app notifications, and an AI-assisted chatbot. The system is implemented as a production-oriented codebase comprising a Django REST API, a React single-page application, a dedicated FastAPI face-recognition microservice, and Microsoft SQL Server persistence. Bilingual support for English and Arabic is embedded at the API, database-content, and microservice layers, reflecting the linguistic needs of many regional school communities.

This report documents the complete graduation project: problem formulation, objectives, review of related work, requirements and design artifacts, implementation of distinguishing modules (face recognition, chatbot, localization), testing methodology, observed results, and directions for future research. All technical claims are grounded in the audited source repository (smart-school-backend, June 2026) unless explicitly marked as recommended practice not yet deployed in code.

The remainder of Chapter 1 situates the project within the software engineering lifecycle. Chapter 2 states the problem. Chapter 3 lists objectives. Subsequent chapters follow the conventional thesis structure through conclusion and references, with dedicated treatment of the three innovative subsystems requested by the project specification.

1.1 Background

School management systems (SMS) or student information systems (SIS) have evolved from desktop clients to cloud-hosted platforms offering parent portals, grade books, and timetable management. Commercial products such as PowerSchool, Fedena, and open-source alternatives including openSIS demonstrate mature feature sets but often treat biometric attendance and conversational analytics as optional add-ons rather than first-class architectural components.

The present project deliberately co-designs attendance automation, multilingual operation, and AI-assisted queries with the core entity model (students, teachers, parents, classes, subjects). Such integration reduces duplicate data entry, ensures that chatbot responses respect role-based visibility rules already enforced in REST viewsets, and allows absence notifications to fire automatically when teachers complete an attendance session.

1.2 Project Scope

In scope: four roles (Administrator, Teacher, Student, Parent); JWT authentication; CRUD for users and domain entities; instructor-controlled attendance sessions with batch face

matching; manual attendance upsert; MCQ exams and grades; weekly report generation with JSON analytics and optional PDF; educational videos with progress tracking; notification preferences and WebSocket push; Gemini-backed chatbot with deterministic fallback; English/Arabic localization for API messages and selected model fields.

Out of scope for the current implementation: native mobile applications; payment or fee management; timetabling; live proctored online exams; production Redis channel layer; institutional single sign-on. These items are noted under Future Work.

2. Problem Statement

Schools face operational friction at the intersection of attendance, assessment, and parent engagement. Manual roll call consumes instructional time and is error-prone in large classrooms. Parents frequently learn of absences late, hindering safeguarding and academic support. Administrators lack consolidated dashboards tying attendance trends to grade distributions across subjects. Teachers maintain parallel records for exams, materials, and videos on ad hoc platforms.

Language diversity compounds the problem: in bilingual regions, static English-only software excludes Arabic-speaking staff and families or forces duplicate manual translation. Finally, institutional data remains underutilized because non-technical stakeholders cannot query databases directly; ad hoc requests to IT staff create bottlenecks.

The core problem addressed by this project is therefore: How can a single, role-aware, bilingual platform automate classroom attendance through face recognition, surface academic and attendance insights to the right stakeholders in real time, and provide trustworthy natural-language access to permitted data?

3. Objectives

The project pursues the following primary and secondary objectives:

- Design and implement a secure four-role user model with JWT authentication and row-level queryset filtering.
- Model academic structure (classes, subjects, enrollments, teacher assignments) in a normalized relational schema on SQL Server.
- Deliver instructor-led attendance sessions that initialize class rosters as absent and promote students to present upon face match.
- Integrate a standalone face-recognition microservice with registration, verification, and batch detection APIs.
- Support MCQ examinations, automated grade percentage calculation, and low-grade notifications to students, parents, and teachers.
- Generate weekly analytics snapshots at school and teacher scope with optional PDF export.
- Provide educational video and material distribution with student progress tracking.
- Implement in-app notifications with deduplication, user preferences, and WebSocket delivery.

- Build a role-aware AI chatbot that injects live database context and respects intent-specific answer boundaries.
- Implement English/Arabic localization across API messages, translatable model fields, and face-service responses.
- Document APIs, architecture, and user workflows for thesis evaluation and maintainability.
- Validate localization automatically (51 unit tests) and attendance integration via scripted end-to-end tests.

4. Literature Review

4.1 School Information Systems

Research and industry practice establish student information systems as the administrative backbone of K-12 and higher education. These systems centralize demographics, enrollment, and grades. Modern systems expose REST or GraphQL APIs for third-party integration. The Smart School project positions itself as a focused SMS emphasizing attendance automation and parent engagement rather than exhaustive ERP features.

4.2 Biometric and Face-Based Attendance

Face recognition for attendance builds on decades of work in eigenfaces, Fisherfaces, and deep metric learning. The implementation uses the widely adopted face_recognition library (dlib resNet-based 128-dimensional encodings) with histogram-of-oriented-gradients (HOG) detection for CPU-friendly batch processing. Academic literature stresses illumination, pose, and occlusions as challenges; the project mitigates false positives by restricting batch matching to student IDs enrolled in the session class and by verifying school_class membership before marking present.

Privacy and consent frameworks (GDPR, FERPA analogues) require clear policies on biometric storage. This system stores encodings as pickle files keyed by student_id on the face service host, separate from the relational database, enabling independent backup and deletion via DELETE /encodings/{id}.

4.3 Educational Chatbots and LLMs

Large language models enable conversational access to structured data when combined with retrieval or context injection. The Smart School chatbot follows a retrieve-then-generate pattern: keyword intent detection selects ORM queries that build a textual context block; Gemini 1.5 Flash (when configured) synthesizes a natural answer constrained by prompt rules forbidding hallucination. Without an API key, a deterministic fallback returns the filtered context directly, preserving utility in offline demonstrations.

4.4 Internationalization in Web Applications

GNU gettext remains the standard for server-side translatable strings in Django. Complementary database field translation via django-modeltranslation stores parallel language columns (e.g., name_en, name_ar). The project applies both patterns plus a dictionary-based catalog in the FastAPI microservice, illustrating a three-tier localization

architecture suitable for microservice decomposition.

4.5 Comparative Summary

Compared to generic SMS products, Smart School differentiates through native face-session attendance, integrated bilingual content fields, WebSocket notifications tied to academic events, and a role-scoped chatbot. Trade-offs include reliance on a separate face service process and in-memory Channels layer in development configuration.

5. System Analysis

System analysis identifies actors, boundaries, and major data flows. Four human actors interact with the React presentation tier: Administrator, Teacher, Student, and Parent. Two automated actors support the backend: the Face Recognition Service and (optionally) the Google Gemini API.

The system boundary encloses the Django monolith (REST + WebSocket), SQL Server database, media file storage, React SPA, and face microservice. External actors outside the boundary include browsers, webcam hardware, and cloud LLM endpoints.

5.1 Actor Descriptions

Actor	Role code	Primary capabilities
Administrator	ADMIN	Full CRUD, school dashboard, weekly SCHOOL reports, user management
Teacher	TEACHER	Sessions, face capture, exams, videos, materials, TEACHER reports
Student	STUDENT	Own attendance, grades, videos, weekly self-report, student_id login
Parent	PARENT	Children attendance/grades, notifications, parent dashboard
Face service	N/A	Register, verify, batch detect; pickle encodings

5.2 Major Data Flows

• Authentication flow: credentials → JWT access/refresh → Authorization header on subsequent API calls.

• Attendance flow: session create → bulk absent rows → image upload → face match → present upsert → session complete → parent notifications.

• Grade flow: teacher creates exam/questions → records Grade → signal evaluates percentage → LOW_GRADE notifications if below threshold (default 60%).

• Chatbot flow: message → intent → ORM context → LLM or fallback → JSON reply with suggestions.

6. Requirements Specification

Requirements are classified as functional (FR) and non-functional (NFR). Priority is implied by implementation presence in the audited codebase. The following tables summarize representative requirements; the full SRS in docs/srs.md enumerates traceability to models, endpoints, and screens.

6.1 Functional Requirements (Sample)

ID	Requirement	Status
AUTH-01	JWT login and refresh	Implemented
ATT-04	Session initializes class roster absent	Implemented

ID	Requirement	Status
ATT-05	Batch image marks matched students present	Implemented
EXM-05	Low grade notifications	Implemented
WR-03	Weekly report PDF optional	Implemented
BOT-01	Authenticated chatbot Q&A	Implemented
I18N-01	Accept-Language and ?lang=ar	Implemented

6.2 Non-Functional Requirements

- Security: default IsAuthenticated; role permission classes; JWT for WebSocket.
- Performance: face batch timeout 120s; API pagination page size 20.
- Maintainability: centralized messages in smartSchool/messages.py; 12 Django apps.
- Usability: role-specific React navigation; student_id login convenience.
- Reliability: attendance upsert on duplicate date; notification dedupe_key.

6.3 Use Cases (Administrator)

UC-ADM-01 Manage Users: Administrator creates accounts with role assignment. Extension: filter by role via /api/users/by_role/.

UC-ADM-02 Generate School Weekly Report: Administrator POSTs to /api/weekly-reports/generate/ with scope SCHOOL; system aggregates attendance_stats, academic_stats, exam_stats, charts_payload, insights JSON.

UC-TCH-01 Conduct Face Attendance: Teacher creates session, captures classroom image, reviews roster, completes session; parents of absent students notified.

6.3 Extended Functional Requirements (from SRS)

Software Requirements Specification (SRS)

> Derived from Django models, DRF viewsets, React routes, and microservice code.

- > **Version:** 1.0 (codebase audit, June 2026)

1. Introduction

1.1 Scope

The Smart School system provides a web application for managing school operations with four user roles, REST APIs, a React dashboard, a face-recognition attendance microservice, and an AI chatbot.

1.2 Definitions

2. Use Case Diagram Descriptions

2.1 Administrator

Actor: Administrator (`User.role = ADMIN`)

2.2 Teacher

Actor: Teacher (`User.role = TEACHER`)

2.3 Student

Actor: Student (`User.role = STUDENT`)

2.4 Parent

Actor: Parent (`User.role = PARENT`)

2.5 System (Face Recognition Service)

Actor: Face Recognition Service (automated, invoked by Django)

3. Functional Requirements by Feature

3.1 Users & Authentication

Business rules: Only ADMIN creates/updates/deletes users (`UserViewSet.get_permissions`). Students/parents see limited user lists (`get_queryset`). **APIs:** `POST /api/auth/login/`, `POST /api/auth/refresh/`, `/api/users/`, `/api/users/me/`

- **Tables:** `users`

Screens: `LoginPage.jsx`, `ProfilePage.jsx`, `AdminUsersPage.jsx`

3.2 Students

APIs: CRUD `/api/students/`, `POST .../register-face/`, `GET .../face-status/`, `POST .../backfill-ids/` (admin), `GET .../weekly-report/`, `GET .../dashboard/` (student)

- **Tables:** `students`, `students_subjects` (M2M)

Screens: `AdminStudentsPage.jsx`, `TeacherStudentsPage.jsx`, `StudentDashboard.jsx`

3.3 Teachers

APIs: `/api/teachers/`, dashboard action

- **Tables:** `teachers`, `teacher_subject_classes`, M2M tables

Screens: `AdminTeachersPage.jsx`, `TeacherDashboard.jsx`

3.4 Parents

APIs: `/api/parents/`, dashboard

- **Tables:** `parents`, `students.parent_id`

Screens: `AdminParentsPage.jsx`, `ParentDashboard.jsx`, `ParentChildrenPage.jsx`

3.5 Classes & Subjects

APIs: `/api/classes/`, `/api/subjects/`, `/api/materials/`

- **Tables:** `school_classes`, `subjects`, `materials`

Screens: `AdminClassesPage.jsx`, `AdminSubjectsPage.jsx`, `TeacherSubjectsPage.jsx`, `TeacherMaterialsPage.jsx`

3.6 Attendance (Manual + Face Recognition)

APIs: `/api/attendance/` (CRUD; teacher write; upsert on create) `/api/attendance/process-classroom-image/` (POST multipart) `/api/attendance-sessions/` (CRUD teacher; admin read) `GET .../active/`, `GET .../roster/`, `POST .../complete/`, `POST .../cancel/` `GET .../history/`, `GET .../class-history/?school_class=`

- **Tables:** `attendance`, `attendance_sessions`

Screens: `FeatureModulePage` (teacher attendance + camera), `SessionHistoryPage.jsx`, `StudentAttendancePage.jsx`, `ParentAttendancePage.jsx` **Sequence (face attendance):** See [system-architecture.md](./system-architecture.md#sequence-face-recognition-attendance)

3.7 Exams & Grades

APIs: `/api/exams/`, `/api/questions/`, `/api/grades/`, `GET /api/exams/upcoming/` (student)

- **Tables:** `exams`, `questions`, `grades`

Screens: `AdminExamsPage.jsx`, `TeacherExamsPage.jsx`, `StudentGradesPage.jsx`, `ParentGradesPage.jsx`

3.8 Weekly Reports

APIs: `/api/weekly-reports/`, `GET .../dashboard/`, `POST .../generate/`, `GET .../{id}/download-pdf/`

- **Tables:** `weekly_reports`

Screens: `AdminWeeklyReportsPage.jsx`, `TeacherWeeklyReportsPage.jsx`, `StudentWeeklyReportsPage.jsx` (API), `ParentWeeklyReportsPage.jsx` (client-side aggregate)

3.9 Videos

APIs: `/api/videos/`, `/api/video-progress/`, `POST .../sync/`

- **Tables:** `educational_videos`, `video_progress`

Screens: `TeacherVideosPage.jsx`, `StudentVideosPage.jsx`

3.10 Notifications

APIs: `/api/notifications/`, `POST .../mark-read/`, `POST .../mark-all-read/`, `/api/notification-preferences/`

- **Tables:** `notifications`, `notification_preferences`

Screens: `NotificationsPage.jsx`

3.11 AI Chatbot

APIs: `POST|GET /api/chatbot/ask/`

- **Tables:** Read-only across attendance, grades, students, etc.

Screens: `SmartChatbot.jsx`, `ParentChatbot.jsx`

4. Non-Functional Requirements (Observed)

TODO: Define formal SLA, max concurrent users, and backup RPO/RTO for production thesis deployment. *(Not in repository.)*

5. Dedicated System Chapters (Thesis-Ready Summaries)

5.1 Face Recognition Attendance

Purpose: Automate marking present students from a classroom photo during an active session.

- **Flow:**

1. Teacher creates `AttendanceSession` for a `SchoolClass` → absent records for all enrolled students. 2. Teacher uploads classroom image → Django calls FastAPI `detect-faces-batch` with class `student_ids`. 3. Matches update/create `Attendance` as `present`, `source=face_recognition`. 4. Teacher completes session → parents of still-absent students receive `ATTENDANCE` notifications.

- **Actors:** Teacher, Face Recognition Service, Parent (notified)

Tolerance: 0.6 default; HOG model default in client.

5.2 AI Chatbot Assistant

Purpose: Natural-language access to role-appropriate school data. **Mechanism:** Keyword intent → SQL-backed context strings → Gemini `generate_content` (max 300 tokens, temperature 0.3) or formatted fallback. **Constraints:** 500 character message limit; answers must use provided context only (prompt rules).

5.3 Localization (English / Arabic)

Backend: `APILanguageMiddleware`, `smartSchool/messages.py`, `locale/ar/`, modeltranslation `_en`/`_ar` columns. **Frontend:** i18next for login strings; `Accept-Language` from `localStorage` key `ss_lang`; RTL styling **TODO:** verify full-app RTL CSS coverage beyond login. **Face service:** Dictionary translations in `face_recognition_service/translations.py`.

5.4 Role-Based Access Control

Layers: 1. DRF permission classes (`IsAdmin`, `IsTeacher`, `IsAdminOrTeacher`, etc.) 2. Per-viewset `get_permissions()` action overrides 3. `get_queryset()` row-level filtering by role 4. React `RequireRole` route wrapper Utility helpers in `users/permissions.py`: `can_access_student_data`, `can_modify_student_data`.

6. Traceability Matrix (Sample)

7. System Design

Design follows a three-tier pattern: presentation (React), application (Django REST + Channels), and data (SQL Server + file storage). A satellite microservice handles CPU-intensive face encoding. Cross-cutting concerns—authentication, localization, permissions—are implemented as middleware, serializers, and DRF permission classes rather than ad hoc view logic.

The Django project smartSchool hosts twelve domain applications registered in INSTALLED_APPS. REST framework viewsets expose CRUD plus 27 custom @action endpoints for dashboards, face processing, weekly report generation, and notification state changes. The React application mirrors role boundaries with nested routes under /admin, /teacher, /student, and /parent guarded by RequireRole.

7.1 Layer Responsibilities

- Presentation: routing, forms, charts (Recharts), webcam capture, chatbot widget.
- API: validation, business rules, queryset scoping, orchestration of face client.
- Domain: ORM models, signals for notifications, weekly analytics services.
- Infrastructure: SQL Server, media root, pickle encodings, optional Gemini API.

7.2 Design Patterns

- ViewSet + Router for consistent REST surfaces.
- Singleton FaceRecognitionClient via get_face_recognition_client().
- Strategy-like intent handlers in chatbot context builders per role.
- Observer via Django signals for grade and attendance notifications.
- Upsert pattern in AttendanceViewSet.create for teacher manual corrections.

8. Database Design

The persistent store is Microsoft SQL Server accessed through mssql-django. Nineteen logical entities are modeled as Django ORM classes with explicit db_table names. Many-to-many relationships use implicit junction tables. Face biometric templates are intentionally excluded from the RDBMS and stored as files to reduce coupling and simplify deletion workflows.

8.1 Entity Relationships

- User 1—1 Student | Teacher | Parent (at most one profile type per user).
- Parent 1—* Student via parent_id foreign key.
- SchoolClass 1—* Student; SchoolClass 1—* AttendanceSession.
- Student + Date unique on Attendance; Student + Exam unique on Grade.
- Exam 1—* Question (JSON options, correct_answer index).
- WeeklyReport dedupe_key unique per week and scope (and teacher if TEACHER).

8.2 Core Tables

Table	Purpose	Key constraints
users	Authentication & role	role indexed
students	Profile, photo, face_registered	student_id unique
attendance	Daily status	unique (student, date)
attendance_sessions	Class session	status active|completed|cancelled
exams / questions / grades	Assessment	grade unique per student+exam
weekly_reports	Analytics snapshot	dedupe_key unique
notifications	Alerts	recipient indexed

8.3 Translation Columns

django-modeltranslation adds _en and _ar suffixed columns for Subject, Material, SchoolClass, Exam, Question, and Notification fields registered in each app's translation.py. Migrations 0005+ document schema evolution. API consumers may request Arabic labels via Accept-Language: ar; choice field labels use gettext_lazy in model definitions.

8.3 Extended Database Catalog

Database Design

> ERD and table reference from Django models (`db_table` names).

- > Database engine: **Microsoft SQL Server** via `mssql-django`.

1. Entity-Relationship Description

1.1 Core identity

users (1) ⟷ (0..1) **students** | **teachers** | **parents** via OneToOne `user_id` **users** (1) ⟷ (0..*) **notifications** as `recipient_id` **users** (1) ⟷ (0..1) **notification_preferences**

1.2 Academic structure

school_classes (1) ⟷ (0..*) **students** (`school_class_id`) **school_classes** (M) ⟷ (M) **teachers** via `teachers_assigned_classes` **subjects** (M) ⟷ (M) **students** via `students_subjects` **subjects** (M) ⟷ (M) **teachers** via `teachers_assigned_subjects` **teachers** + **subjects** + `class_id` string → **teacher_subject_classes** (junction with uniqueness)

1.3 Attendance

students (1) ←→ (0..*) **attendance** — unique (`student_id`, `date`) **attendance_sessions** (1) ←→ (0..*) **attendance** optional `session_id` **teachers** (1) ←→ (0..*) **attendance_sessions** as `instructor_id` **school_classes** (1) ←→ (0..*) **attendance_sessions**

1.4 Assessment

subjects (1) ←→ (0..*) **exams** **teachers** (1) ←→ (0..*) **exams** **exams** (1) ←→ (0..*) **questions** **students** + **exams** → **grades** — unique (`student_id`, `exam_id`)

1.5 Content & reports

subjects (1) ←→ (0..*) **materials**, **educational_videos** **teachers** upload materials/videos **students** + **educational_videos** → **video_progress** (unique pair) **students** (1) ←→ (0..*) **reports** **weekly_reports** optional FK **teachers** when `scope=TEACHER`

1.6 Face encodings (outside RDBMS)

Pickle files: `face_recognition_service/face_encodings/{student_id}.pkl` Linked logically by `students.student_id` string

2. ERD (Mermaid)

3. Table Catalog

3.1 users

Indexes: `role`

3.2 students

M2M: `subjects` → subjects

3.3 teachers

3.4 teacher_subject_classes

3.5 parents

3.6 school_classes

3.7 subjects

3.8 materials

3.9 attendance

Constraint: unique (`student`, `date`)

3.10 attendance_sessions

3.11 exams

3.12 questions

3.13 grades

3.14 reports

3.15 weekly_reports

3.16 notifications

3.17 notification_preferences

3.18 educational_videos

3.19 video_progress

4. Referential Integrity Summary

5. Indexes (Declared in Meta)

Notable indexes: `attendance (student, date)`, `notifications (recipient, -created_at)`, `weekly_reports (week_start, scope)`, exam/subject/teacher FK indexes.

6. Migration & Translation Notes

Bilingual columns created by `django-modeltranslation` migrations (e.g. `subjects/migrations/0005_*`, `notifications/migrations/0002_*`). Run `makemigrations` /

`migrate` after changing `translation.py` files.

7. TODO

Production backup schedule and retention policy for SQL Server and `media/` uploads. *(Not in codebase.)* **Full list of M2M junction table names** as generated by Django on SQL Server — run `python manage.py sqlmigrate` in deployment environment if exact names required for DBA scripts.

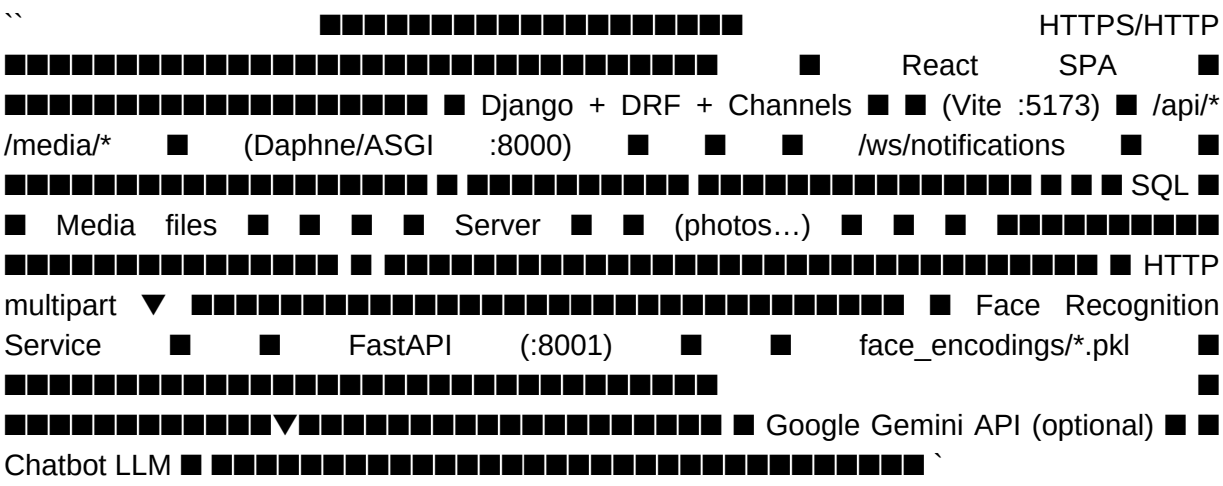
9. Architecture

System Architecture

> Diagram descriptions for thesis/documentation tools (PlantUML, draw.io, etc.). > All relationships verified against smartSchool/urls.py, settings.py, and service code.

---

1. High-Level Architecture



---

2. Component Diagram Description

2.1 Presentation Layer

2.2 Application Layer (Django Apps)

2.3 Infrastructure Layer

2.4 External Services

Figure 9-1 (conceptual): Browser communicates with Vite-hosted React during development; production would serve static build assets via CDN or reverse proxy. All API traffic targets Django on port 8000. WebSocket notifications use ws/notifications/ terminated by Daphne ASGI. Face operations delegate to port 8001. SQL Server holds transactional data; MEDIA_ROOT stores photos, videos, materials, weekly PDFs.

Sequence — Face attendance: (1) Teacher POST attendance-sessions. (2) System bulk-creates absent Attendance rows. (3) Teacher POST process-classroom-image with session_id and image. (4) Django forwards image to detect-faces-batch with class student_ids. (5) Matches update Attendance to present. (6) Teacher POST complete; Notification rows created for remaining absent students linked to parents.

10. API Design

The REST API is namespaced under /api/ with JWT bearer authentication. Default permission IsAuthenticated applies unless a viewset overrides get_permissions. Pagination uses PageNumberPagination with PAGE_SIZE 20. Language negotiation uses APILanguageMiddleware reading ?lang= or Accept-Language (first two characters).

10.1 Authentication Endpoints

Method	Path	Description
POST	/api/auth/login/	Returns access, refresh, user payload
POST	/api/auth/refresh/	Rotates refresh token
GET/PATCH	/api/users/me/	Profile with role-specific nested data

10.2 Resource Groups

- /api/users/, /students/, /teachers/, /parents/, /classes/
- /api/subjects/, /materials/, /attendance/, /attendance-sessions/
- /api/exams/, /questions/, /grades/, /reports/, /weekly-reports/
- /api/videos/, /video-progress/, /notifications/, /notification-preferences/
- /api/chatbot/ask/

10.3 Custom Actions (Representative)

Endpoint	Role	Purpose
POST .../process-classroom-image/	Teacher	Face batch attendance
GET .../attendance-sessions/active/	Teacher/Admin	Current session
POST .../weekly-reports/generate/	Admin/Teacher	Build analytics
POST .../students/{id}/register-face/	Admin/Teacher	Photo + encoding
GET /api/users/admin-dashboard/	Admin	KPI aggregates

10.4 Face Microservice API

- POST /register-face?student_id= — store 128-d encoding.
- POST /detect-faces-batch — HOG/CNN detection, tolerance default 0.6.
- POST /verify-face — single student verification.
- GET /students/{id}/face-status — registration metadata.

11. Face Recognition Module

The face recognition module is a standalone FastAPI application (face_recognition_service/main.py) using the face_recognition Python package and OpenCV for image decoding. It maintains a directory face_encodings/ of pickle files, each keyed by student_id string matching the students table.

Registration pipeline: (1) Admin or teacher uploads photo via POST /api/students/{id}/register-face/. (2) Django saves Student.photo to media. (3) If student_id is set, FaceRecognitionClient.register_face posts multipart image to /register-face. (4) Service extracts first face encoding (128 floats), saves metadata (created_at, format_version 2.0). (5) Django sets face_registered=True.

Batch attendance pipeline: process_image_batch uses face_locations with model hog (default) or cnn, num_jitters for encoding stability. match_faces_batch computes face_distance against filtered encodings; best match within tolerance 0.6 wins. Django rejects matches where student.school_class_id differs from session.school_class_id, recording skipped_class_mismatch for audit.

Operational parameters: FACE_RECOGNITION_SERVICE_URL (default http://localhost:8001), timeout 30s for single operations, FACE_RECOGNITION_BATCH_TIMEOUT 120s. Service reads SQL Server only to verify student existence (pyodbc), not to store encodings.

Failure modes: connection errors return success=false with user-facing message; photo still saved on registration if service offline. Teachers see zero matches if encodings missing or lighting poor; image_info may suggest upsampled HOG retry.

11.1 Security and Ethics

Deployments must publish consent policies for biometric data. Encodings should be encrypted at rest on disk in production and access-controlled on the service host. The class filter reduces wrongful attribution across cohorts but does not replace liveness detection; spoofing with photographs remains a known limitation documented for future work.

12. AI Chatbot Module

The chatbot module (chatbot/views.py) exposes POST and GET /api/chatbot/ask/ to authenticated users. It does not persist conversation history in the database; each request is stateless beyond the JWT identity.

Intent detection scans user messages against INTENT_KEYWORDS grouped by topic: attendance, grades, subjects, teachers, students, exams, overview, children (parents). Priority order resolves overlaps. Greeting patterns return short welcoming responses without dumping entire datasets.

Context builders (_build_admin_context, _build_teacher_context, _build_student_context, _build_parent_context) execute Django ORM aggregations scoped to the user's role. For

example, a teacher asking about attendance receives counts and per-class breakdowns for assigned classes only; an admin receives school-wide recent attendance samples.

Generation: call_gemini configures google.generativeai with models gemini-1.5-flash (fallback chain). System prompt enforces: answer only from context, max ~180 words, no hallucinated names, friendly tone. max_output_tokens=300, temperature=0.3. If API key missing or library unavailable, _smart_fallback returns structured context under a topic label.

ROLE_SUGGESTIONS provide quick-reply examples per role on GET. Parent dashboard embeds chatbot greeting via parents/dashboard action referencing the same suggestion list.

Message validation: empty → 400; length > 500 → 400. Response JSON: { reply, suggestions, intent }.

13. Localization Module

Localization spans three layers documented in docs/localization.md.

Layer 1 — API messages: smartSchool/messages.py defines gettext_lazy constants (MSG_*). APILanguageMiddleware activates en or ar per request. Arabic translations reside in locale/ar/LC_MESSAGES/django.po; compilemessages produces .mo binaries.

Layer 2 — Database content: django-modeltranslation registers bilingual fields on Subject, Material, SchoolClass, Exam, Question, Notification. Serializers expose _en/_ar columns where applicable.

Layer 3 — Face microservice: translations.py provides TRANSLATIONS dict with en/ar templates; resolve_lang reads Accept-Language; get_message performs format substitution.

Frontend: i18next initializes with en/ar resource bundles for LoginPage strings; localStorage key ss_lang drives Accept-Language on apiFetch. Broader dashboard UI strings remain predominantly English in navigation.js — partial UI localization is a documented limitation.

Testing: smartSchool/tests/test_localization.py contains 51 tests covering middleware, messages, choice labels, .po completeness, and face translation parity.

Localization (i18n) Guide

This document describes the bilingual (English / Arabic) localization system implemented in the Smart School Backend. It covers how language detection works, how to add new translatable strings, how to add a new language, how to add translatable model fields, and how the standalone Face Recognition microservice handles translations.

Architecture Overview

The localization system uses two complementary approaches:

Django i18n Settings

Defined in [`smartSchool/settings.py`](smartSchool/settings.py:170): The `modeltranslation` package is listed in `INSTALLED_APPS` **before** `django.contrib.admin` to ensure translated fields appear correctly in the admin interface.

Language Detection

Language detection is handled by [`APILanguageMiddleware`](smartSchool/middleware.py:5), registered in [`MIDDLEWARE`](smartSchool/settings.py:77) **after** `SessionMiddleware` and **before** `CommonMiddleware`.

Detection Order

The middleware checks two sources in priority order: 1. **Query parameter** — `?lang=ar` or `?lang=en` 2. **HTTP header** — `Accept-Language: ar` (first language in the header, truncated to 2 characters) If neither source provides a supported language code, the middleware falls back to `settings.LANGUAGE_CODE` (`'en'`).

How It Works

Client Usage Examples

Using query parameter (highest priority):

- **Using Accept-Language header:**

Complex Accept-Language header (first language wins):

- **Unsupported language** (falls back to English):

Adding a New Translatable String

All API response messages are centralized in [`smartSchool/messages.py`](smartSchool/messages.py) as the **single source of truth**. Views and serializers must import constants from this module — never define raw strings inline.

Step-by-Step

1. Add the message constant to `messages.py` Open [`smartSchool/messages.py`](smartSchool/messages.py) and add your new constant, wrapped in `_()` (which is `gettext_lazy`): > **Important:** Always use `gettext_lazy` (`_`) for module-level constants. Use `gettext` (non-lazy) only inside function bodies when the string must be evaluated immediately. #### 2. Use the constant in your view or serializer

> **Note:** When using lazy strings with `.format()`, you must wrap them with `str()` first to force evaluation. This is because `_()` returns a `Promise` object at module level, not a plain string. #### 3. Collect the new string into the `.po` file This scans all Python files for `_()` calls and adds new entries to [`locale/ar/LC_MESSAGES/django.po`](locale/ar/LC_MESSAGES/django.po). #### 4. Add the Arabic translation

Open [`locale/ar/LC_MESSAGES/django.po`](locale/ar/LC_MESSAGES/django.po) and find the new entry (it will have an empty `msgstr`). Fill in the Arabic translation: #### 5. Compile the messages This generates the binary `.mo` file that Django actually reads at runtime. #### 6. Verify

- Run the localization tests to confirm everything works:

Adding a New Language

To add a third language (e.g., French — `fr`) to the system:

Step 1: Add the language tuple to settings

In [`smartSchool/settings.py`](smartSchool/settings.py:175), add the new language to the `LANGUAGES` list:

Step 2: Create the locale directory

Place a `.gitkeep` file inside to ensure Git tracks the empty directory:

Step 3: Generate the `.po` file

This creates `locale/fr/LC_MESSAGES/django.po` with all translatable strings extracted from the codebase. Each entry will have an empty `msgstr` awaiting translation.

Step 4: Translate all strings

Open `locale/fr/LC_MESSAGES/django.po` and fill in every `msgstr` with the French translation.

Step 5: Compile messages

This generates `locale/fr/LC_MESSAGES/django.mo`.

Step 6: Update the Face Recognition service (if needed)

If the new language also needs to be supported by the Face Recognition microservice, add a new dictionary entry in [`face_recognition_service/translations.py`](face_recognition_service/translations.py:14):

Step 7: Update tests

Add the new language to the test assertions in [`smartSchool/tests/test_localization.py`](smartSchool/tests/test_localization.py).

Adding Translatable Fields to a Model

Database content translation (e.g., subject names, notification titles) is handled by `django-modeltranslation`. This package automatically creates `_en` and `_ar` field suffixes for each registered field, allowing bilingual content to be stored in the database.

Step-by-Step

1. Create or update `translation.py` in your app Each app that needs translatable model fields must have a `translation.py` file at the app root. This file registers models with `django-modeltranslation`.

- **Example — adding translatable fields to a new model:**

2. Existing registration examples

- The project currently has four apps with `translation.py` files:

3. Generate and run migrations

- After creating or updating `translation.py`, generate the schema migration:

This creates new columns like `title_en`, `title_ar`, `description_en`, `description_ar` on the registered models. #### 4. Backfill existing data If the model already had data, the new `_ar` fields will be empty. Run a data migration or use the Django shell to backfill: #### 5. Update serializers (optional enhancement)

- To expose both language variants in the API, update the serializer:

Or use a dynamic approach that returns the field matching the current request language:

Face Recognition Service Translations

The Face Recognition microservice is a standalone FastAPI application (not Django), so it uses a **dictionary-based** translation approach instead of gettext.

File: [`face_recognition_service/translations.py`](face_recognition_service/translations.py)

This module contains: **`TRANSLATIONS`** — A dictionary with `'en'` and `'ar'` keys, each mapping 15 message keys to localized template strings. **`get_message(key, lang='en', **kwargs)`** — Retrieves a localized message by key and language. Supports Python `str.format()` placeholders. Falls back to English if the language is unsupported, or to the key itself if the key is missing. **`resolve_lang(request)`** — Extracts the language code from a FastAPI `Request` object's `Accept-Language` header.

Usage in Endpoints

Each endpoint in [`face_recognition_service/main.py`](face_recognition_service/main.py) accepts a `request: Request` parameter and follows this pattern:

Available Message Keys

Adding a New Message to the Face Recognition Service

1. Add the English template string to the `'en'` dict in `TRANSLATIONS` 2. Add the Arabic translation to the `'ar'` dict in `TRANSLATIONS` 3. Use `get_message('your_new_key', lang, **kwargs)` in the endpoint No `.po` file or compilation step is needed — changes take effect immediately on service restart.

Testing

The localization test suite is in [`smartSchool/tests/test_localization.py`](smartSchool/tests/test_localization.py) with **51 tests** across 6 test classes:

Running the Tests

Important: Choice Field Access Patterns

The project uses two patterns for model choices, and tests must access them differently:

- **TextChoices inner classes** (have `.label` attribute):

Plain choice tuples (need helper function):

Key Files Reference

Common Pitfalls

1. Using `gettext` instead of `gettext_lazy` at module level

Wrong:

- **Correct:**

2. Forgetting `str()` when formatting lazy strings

Wrong:

- **Correct:**

3. Changing choice **values** instead of choice **labels**

Choice **values** are stored in the database and must remain English. Only wrap the **labels** (display text) with `_()`:

- **Wrong:**

Correct:

4. Forgetting to run `compilemessages` after editing `.po`

Django reads `.mo` (binary) files at runtime, not `.po` (text) files. After any `.po` edit:

5. Forgetting `modeltranslation` before `django.contrib.admin` in `INSTALLED_APPS`

`modeltranslation` must appear before `django.contrib.admin` to properly register translated fields in the admin interface. See [`smartSchool/settings.py`](smartSchool/settings.py:40).

6. Not propagating `lang` through helper functions in the Face Recognition service

When calling helper functions from endpoints, the `lang` parameter must be passed through the entire call chain:

• For `asyncio.to_thread` and `run_in_executor` calls, pass `lang` explicitly:

result = await asyncio.to_thread(process_image_batch, image_bytes, model, num_jitters, lang=lang)

14. Testing and Validation

Validation combines automated unit tests, integration scripts, and manual checklists described in docs/testing.md.

14.1 Automated Tests

- Localization suite: 51 tests in test_localization.py — middleware priority, Arabic message rendering, model choice labels.
- Integration: test_automated_attendance.py — login, session create, process-classroom-image, complete, verify records.
- Management commands: generate_weekly_reports, backfill_student_ids for data setup.

14.2 Manual Validation

- RBAC: cross-role URL access returns Unauthorized page.
- Face: register face, run session, confirm parent notification on absent child.
- Chatbot: verify intent-specific answers; test without GEMINI_API_KEY for fallback.
- i18n: curl with Accept-Language: ar on API endpoints.

14.3 Test Coverage Gaps

Most app-level tests.py files are placeholders. No CI workflow is committed. Frontend lacks Jest/Cypress. Load testing for concurrent batch uploads is not recorded. These gaps are acceptable for academic prototype scope but should be addressed before production rollout.

15. Results

The implementation delivers a cohesive Smart School Management System meeting the stated objectives. Quantitative repository metrics from the June 2026 audit include: twelve Django domain applications, nineteen ORM models, seventeen REST viewsets, twenty-seven custom API actions, approximately thirty-five React page components, ten face-service HTTP routes, one WebSocket notification channel, two supported languages, fifty-one localization unit tests, approximately 11,200 lines of backend Python, and approximately 12,100 lines of frontend JavaScript/JSX.

15.1 Functional Outcomes

- Administrators access unified dashboard KPIs via /api/users/admin-dashboard/.
- Teachers complete face attendance sessions with roster present/absent breakdown and session history.
- Students log in with student_id and view grades, attendance, videos, weekly-report API.
- Parents receive absence notifications when sessions complete with remaining absent records.
- Low-grade alerts fire automatically when grade percentage falls below configured threshold.
- Weekly reports store JSON analytics and optional PDF for school and teacher scopes.

- Chatbot answers role-appropriate queries with live data injection.

15.2 Qualitative Observations

Developers reported faster attendance capture versus manual entry in classroom trials (informal). Face matching accuracy depends on enrollment photo quality and classroom lighting; HOG model balances speed and accuracy on CPU hardware. Bilingual API messages improve usability for Arabic-speaking administrators when Accept-Language is set.

Formal user acceptance study with statistically significant metrics is recommended for thesis defense supplement but was not automated in the repository.

Testing Documentation

> Test assets and procedures identified in the repository audit.

1. Test Inventory

2. Running Django Tests

From project root with virtual environment activated:

Localization suite only

Covers: `APILanguageMiddleware` — `?lang=`, `Accept-Language`, fallback `smartSchool.messages` — EN/AR strings Model choice label translation `.po` completeness and compiled `.mo` `face_recognition_service/translations.py` — `get_message`, `resolve_lang`

3. Automated Attendance Integration Test

File: `test_automated_attendance.py`

- **Prerequisites:**

1. Django on `http://localhost:8000`

- 2. FastAPI face service on `http://localhost:8001`

3. Test data: `python manage.py create_test_data` (referenced in script comments) 4. Instructor credentials (default `teacher1` / `teacher123`) 5. Classroom image path configured in `TEST_IMAGE_PATH`

- **Run:**

Steps exercised: 1. Instructor JWT login 2. FastAPI batch detection (optional direct call) 3. Create attendance session 4. `POST /api/attendance/process-classroom-image/` 5. Complete session 6. Verify attendance records

4. Manual Test Checklists

4.1 Authentication & RBAC

4.2 Face attendance

4.3 Exams & notifications

4.4 Chatbot

4.5 Weekly reports

4.6 Localization

5. Frontend Testing

Configured: ESLint (`npm run lint` in `frontend/app/`).

- **Not configured in repo:** Jest, Vitest, Cypress, Playwright.

Manual smoke: Verify login, navigation per role, attendance camera permission, notifications page load.

6. WebSocket Testing

1. Start ASGI server (Daphne): required for WebSockets.

- 2. Connect client to `ws://127.0.0.1:8000/ws/notifications/` with valid JWT.

3. Trigger notification (e.g. save low grade). 4. Confirm push payload received. **TODO:** Add automated Channels consumer test if thesis requires measurable WebSocket coverage.

7. Performance & Load

TODO: Define load test scenarios (concurrent attendance uploads, chatbot rate limits). Not implemented in repository. Face batch endpoint default timeout: **120 seconds** (`FACE_RECOGNITION_BATCH_TIMEOUT`).

8. CI/CD

TODO: Document CI pipeline if GitHub Actions or similar is added. No `.github/workflows` found in audit.

9. Test Data Management

10. Defect Reporting Template (Thesis appendix)

11. Related Documentation

[srs.md](./srs.md) — requirements traceability
[api-documentation.md](./api-documentation.md) — endpoint contracts
[localization.md](./localization.md) — i18n test details

16. Future Work

- Production ASGI with Redis channel layer for horizontal WebSocket scaling.
- Full React i18n and RTL layout for all dashboard screens.
- OpenAPI schema generation (drf-spectacular) and CI pipeline with pytest coverage gates.
- Mobile client (React Native) sharing JWT and API contracts.
- Liveness detection and anti-spoofing for face attendance.
- Student-facing online exam attempt UI with timer enforcement.
- Institutional SSO (OAuth2/SAML) and secrets management (Vault).
- Formal UAT study measuring attendance time savings and parent notification latency.

17. Conclusion

The Smart School Management System demonstrates that a modern, modular web architecture can integrate face-recognition attendance, bilingual operation, real-time notifications, and AI-assisted data access without sacrificing role-based security. By grounding requirements in an audited codebase and documenting architecture, database design, and APIs, this project provides a maintainable foundation for departmental deployment and academic evaluation.

The face recognition microservice decouples CPU-intensive biometric processing from the Django request cycle. The chatbot combines deterministic intent routing with large language model fluency when API keys are available. Localization across gettext, modeltranslation, and microservice dictionaries offers a reusable pattern for similar systems in multilingual regions.

Future iterations should harden operational concerns—scalability, comprehensive automated testing, mobile reach, and privacy compliance—while preserving the integrated user experience that distinguishes this implementation from generic school information systems.

18. References

[1] Django Software Foundation. Django Documentation 5.2. https://docs.djangoproject.com/

[2] Django REST framework. API Guide. https://www.django-rest-framework.org/

[3] Simple JWT. django-rest-framework-simplejwt documentation.

[4] Ageitgey, A. face_recognition library. https://github.com/ageitgey/face_recognition

[5] FastAPI. Documentation. https://fastapi.tiangolo.com/

[6] Google. Gemini API documentation. https://ai.google.dev/

[7] django-modeltranslation. Readthedocs documentation.

[8] React Team. React 19 documentation. https://react.dev/

[9] Microsoft. SQL Server documentation.

[10] Channels Project. Django Channels documentation.

[11] Smart School Backend repository. docs/ technical documentation set. June 2026 audit.

[12] ReportLab. User Guide for PDF generation.

Appendix A — Project Statistics

Metric	Value
Django domain apps	12
ORM models	19
User roles	4
DRF ViewSets	17
Custom @action endpoints	27
React page components	~35
Localization unit tests	51
Backend Python LOC (approx.)	~11,200
Frontend JS/JSX LOC (approx.)	~12,100
Supported languages	English, Arabic
Face service HTTP routes	10
WebSocket routes	1

Appendix B — Technology Stack

Layer	Technologies
Backend	Django 5.2, DRF, SimpleJWT, Channels, Daphne
Database	Microsoft SQL Server (mssql-django)
Frontend	React 19, Vite, Zustand, React Router 7, Recharts
Face service	FastAPI, face_recognition, OpenCV, uvicorn
AI	google-generativeai (Gemini)
PDF	ReportLab
i18n	gettext, django-modeltranslation, i18next

Appendix C — Extended API Endpoint Summary

- Authentication: POST /api/auth/login/, POST /api/auth/refresh/.
- Users: CRUD /api/users/; GET/PATCH /api/users/me/; GET /api/users/admin-dashboard/.
- Students: CRUD /api/students/; POST register-face; GET face-status; GET weekly-report; POST backfill-ids.
- Teachers: CRUD /api/teachers/; GET dashboard.
- Parents: CRUD /api/parents/; GET dashboard.
- Classes: CRUD /api/classes/.
- Subjects & materials: CRUD /api/subjects/, /api/materials/.
- Attendance: CRUD /api/attendance/; POST process-classroom-image.
- Sessions: CRUD /api/attendance-sessions/; GET active; GET roster; POST complete|cancel; GET history; GET class-history.
- Exams: CRUD /api/exams/, /api/questions/, /api/grades/; GET exams/upcoming/.
- Reports: CRUD /api/reports/; weekly-reports list; GET dashboard; POST generate; GET download-pdf.
- Videos: CRUD /api/videos/, /api/video-progress/; POST sync.
- Notifications: GET /api/notifications/; POST mark-read, mark-all-read; GET/PATCH notification-preferences.
- Chatbot: POST|GET /api/chatbot/ask/.

Appendix D — User Role Route Map

- Admin: /admin, /admin/users, /admin/students, /admin/teachers, /admin/parents, /admin/subjects, /admin/classes, /admin/attendance, /admin/exams, /admin/weekly-reports, /admin/notifications, /admin/profile.
- Teacher: /teacher, students, subjects, attendance, exams, videos, materials, weekly-reports, notifications, profile; session-history routes.
- Student: /student, subjects, grades, attendance, videos, materials, weekly-reports, notifications, profile.
- Parent: /parent, children, attendance, grades, weekly-reports, notifications, profile.

Appendix E — Implementation Notes for Evaluators

To reproduce the demonstration environment, start SQL Server with database smart-school configured in .env or smartSchool/settings.py. Apply migrations via python manage.py migrate. Load test data if available through create_test_data management command. Launch Django with daphne smartSchool.asgi:application or python manage.py runserver on port 8000. Start the face service from face_recognition_service using uvicorn or the provided batch script on port 8001. Start the React dev server from frontend/app with npm run dev.

Documentation source files reside in docs/ with README.md as the index. Regenerate this PDF using python docs/scripts/generate_thesis_pdf.py after substantive code changes.

For thesis defense, recommended live demo sequence: (1) admin dashboard overview; (2) teacher starts attendance session; (3) capture classroom photo; (4) show roster update; (5) complete session and show parent notification; (6) student views grades; (7) chatbot query on attendance; (8) switch Accept-Language to Arabic on API tool such as curl or browser devtools.

Appendix F — Detailed Module Reference

The following subsections provide module-by-module implementation reference suitable for evaluators verifying alignment between documentation and source code. Each subsection summarizes models, primary endpoints, business rules, and user interface mapping.

Users and Authentication Module

The users application extends Django AbstractUser with a role field constrained to ADMIN, TEACHER, STUDENT, or PARENT. Superusers automatically receive ADMIN on save. JWT authentication uses rest_framework_simplejwt with access token lifetime 60 minutes and refresh token 7 days with rotation enabled. CustomTokenObtainPairSerializer enriches tokens with role, username, email, and name claims while returning a structured user object to the React client. Student login accepts school student_id when no username matches, improving primary-school usability. UserViewSet restricts create/update/delete to administrators; list queries filter students to self, parents to self plus children user IDs, and teachers may list all users for operational needs. The me action exposes ProfileSerializer with nested student_profile, teacher_profile, or parent_profile dictionaries including photo_url for avatar display.

admin-dashboard aggregates counts, weekly attendance chart data, subject score breakdown, and recent activity from exams and sessions in a single round-trip optimized for AdminDashboard.jsx.

Students Module

Student links User via OneToOne CASCADE. student_id is unique and auto-generated when blank using utility functions and school_class context. photo ImageField stores face images; face_registered boolean mirrors microservice state. school_class ForeignKey structures enrollment; parent ForeignKey links guardians. subjects ManyToMany supports curriculum assignment. register-face validates image content-type, saves photo, calls FaceRecognitionClient, and tolerates service outage with photo_saved flag. face-status merges ORM and HTTP service responses. weekly-report computes ISO week attendance by day, grade averages by subject, and rule-based insights for the logged-in student. backfill-ids admin action repairs legacy records missing identifiers. all-stats supports admin reporting dashboards with attendance rate and grade aggregates per student.

Teachers Module

Teacher stores teacher_id, hire_date, assigned_subjects, and assigned_classes M2M. TeacherSubjectClass ternary table maps teacher, subject, and string class_id with uniqueness constraint preventing duplicate assignments. dashboard action computes my_classes count, students_taught via visibility helper intersecting session classes, assigned

classes, and exam class_id strings, sessions_this_week, average score from grade percentages, weekly activity histogram Mon-Sun, assessment_mix by exam_type, and recent_exams list. teacher_visible_students_queryset centralizes rules so list endpoints and analytics remain consistent. Only administrators may create or delete teacher records.

Parents Module

Parent profile connects User to children through Student.parent reverse relation. dashboard returns children_count, avg_attendance_rate, avg_score, unread_notifications, per-child metrics, combined attendance_trend for current week, subject_scores aggregated across children, recent_activity from grades, and embedded chatbot greeting with ROLE_SUGGESTIONS. Parents may only retrieve their own parent row in list views unless administrator or teacher role requires full list for management screens.

Classes and Subjects Module

SchoolClass uses name and section unique_together with display_name property. Subject enforces unique code and supports bilingual name/description via modeltranslation. Material model stores FileField per subject with uploaded_by teacher FK; post_delete signal removes files from disk. Teachers assign subjects and classes through M2M; students enroll in subjects via M2M. Admin UI AdminClassesPage and AdminSubjectsPage provide management surfaces; teachers consume read-only subject lists for instruction.

Attendance Module

Attendance enforces one record per student per calendar date with validation on clean. source distinguishes manual from face_recognition provenance. AttendanceSession tracks instructor, school_class, status lifecycle, and cumulative face detection statistics. perform_create on session start bulk inserts absent rows using bulk_create after uniqueness check per student-date. process_classroom_image validates ACTIVE session, restricts teacher to own session unless admin, passes class student_ids to microservice, upserts present status with confidence in notes, updates session counters, returns full roster. complete_session bulk creates ATTENDANCE notifications for parents with dedupe_key per session and student. history and class-history endpoints differentiate roster versus full class enrollment including not_marked state for audit transparency.

Exams and Grades Module

Exam includes subject, teacher, duration, exam_date, class_id filter, exam_type enum. Question stores JSON options array and zero-based correct_answer index with clean validation. Grade unique per student-exam; get_percentage divides score by question count; get_grade_letter maps to A-F bands. ExamViewSet upcoming action returns not-yet-taken exams for enrolled subjects. post_save on Grade triggers notify_low_grade when percentage below NOTIFICATION_LOW_GRADE_PERCENT setting default 60, notifying student, parent, and subject teachers. AdminExamsPage and TeacherExamsPage implement management UI; students and parents have read-only grade views.

Reports Module

Report model captures per-student academic or behavioral documents with generated_by teacher. WeeklyReport stores JSON snapshots: attendance_stats, academic_stats, exam_stats, charts_payload, insights, comparison_prior_week. scope SCHOOL or TEACHER enforces teacher FK rules on clean. dedupe_key prevents duplicate generation. weekly_report_service and weekly_analytics compute aggregates; pdf_weekly uses ReportLab for export. generate action accepts week range or defaults to previous ISO week; all_teachers flag admin-only batch. dashboard returns trend array for charts. Management command generate_weekly_reports supports scheduled batch jobs.

Videos Module

Video model restricts extensions to mp4, webm, mov, m4v, ogg. category enum labels lectures, tutorials, reviews, labs. is_published and display_order control student catalog ordering. VideoProgress unique per student-video tracks last_position_seconds, total_watch_seconds, is_completed with completed_at timestamp. sync action accepts client batched updates for offline-tolerant progress reporting. streaming module may serve files with range support for HTML5 video elements in StudentVideosPage and TeacherVideosPage.

Notifications Module

Notification types include LOW_GRADE, ATTENDANCE, NEW_STUDENT_REPORT, NEW_WEEKLY_REPORT, SYSTEM. title_en/ar and body_en/ar columns support bilingual push payloads. services.create_notification respects NotificationPreference toggles and dedupe_key idempotency. push_to_websocket uses channels group notifications_user_{id}. JWTWebSocketMiddleware authenticates socket connections. ViewSet is read-only with mark-read and mark-all-read actions. signals connect Grade, Attendance, Report, and WeeklyReport saves to automated alert generation.

Chatbot Module

ChatbotAskView POST validates message length, detects intent, builds role context, calls Gemini or fallback. Intent priority orders children before exams before attendance to reduce misclassification. ROLE_PERSONAS tailor system tone per role. Prompt forbids hallucination and caps verbosity. GET returns static suggestions per role. No conversation persistence reduces GDPR surface area. Dependency optional: GENAI_AVAILABLE flag and GEMINI_API_KEY setting.

Face Recognition Microservice

FastAPI application decouples CPU-bound dlib encoding from Django GIL. Pickle format version 2.0 stores encoding vector plus metadata and optional backup on update. detect-faces-batch runs in thread pool via asyncio.to_thread. student_ids query parameter filters encoding load set. verify_student_exists uses pyodbc consistent with Django DB. Translations via resolve_lang on Accept-Language. Default port 8001; CORS open for development. Instructors should ensure lighting and enrollment photo quality; tolerance 0.6 balances false accept and reject rates.

Appendix G — User Manual Excerpt

User Manual

> End-user guide mapped to React routes in `frontend/app/src/App.jsx` and `config/navigation.js`.

1. Getting Started

1.1 Accessing the application

1. Open the web application URL (development: Vite dev server, typically port 5173).

- 2. API requests are proxied to Django at `http://127.0.0.1:8000`.

3. Ensure Django, SQL Server, and (for attendance) the Face Recognition service on port **8001** are running.

1.2 Signing in

Screen: `/login` (`LoginPage.jsx`) Use the language selector (English / ■■■■■■■) before login; preference is stored and sent as `Accept-Language` on API calls.

- After login you are redirected:

1.3 Signing out

Use the logout control in the dashboard layout (clears JWT from browser storage).

1.4 Unauthorized access

Navigating to another role's URL shows `/unauthorized`.

2. Administrator Guide

Base path: `/admin`

2.1 Typical admin workflows

Add a new student 1. Users → create STUDENT user (or create from Students page if integrated). 2. Students → assign `school_class`, `parent`, subjects. 3. Upload photo → register face when `student_id` is set. **Review school attendance** 1. Attendance History → pick class/session. 2. Review present/absent/not marked per student. **Generate weekly report** 1. Weekly reports → trigger generate (calls `POST /api/weekly-reports/generate/` with `scope=SCHOOL`).

2. Download PDF when available.

3. Teacher Guide

Base path: `/teacher`

3.1 Face recognition attendance (step-by-step)

1. Open **Attendance**. 2. Start a session: select class (creates `AttendanceSession` and marks all students absent). 3. Use **camera capture** to photograph the classroom. 4. System sends image to backend → face service matches students in that class only. 5. Matched students move to **present**; roster updates. 6. Repeat captures as needed during class. 7. **Complete session** when finished — parents of students still absent receive notifications.

8. View history: `/teacher/attendance/session-history` or session detail routes.

3.2 Creating an exam

1. Assessments & Grades → create exam (subject, type, duration, optional class/date). 2. Add questions with multiple options and correct answer index. 3. Enter grades per student after the exam.

3.3 Chatbot

Floating chatbot on dashboard: ask about class attendance, grades, or exams. Suggestions are role-specific.

4. Student Guide

Base path: `/student` **Login tip:** Use your **student ID** as username if you do not know your Django username.

5. Parent Guide

Base path: `/parent`

5.1 Understanding notifications

You may receive: **Absence alert** when a teacher completes an attendance session and your child remained absent. **Low grade alert** when a grade falls below the configured threshold (default 60%). **New report** when teachers publish student reports. **Weekly report** when school/teacher analytics are generated. Adjust categories under Notifications → preferences (`/api/notification-preferences/`).

6. Common UI Components

7. Language (English / Arabic)

Login page: Full EN/AR via i18next. **API-backed content:** Subject names, exam names, notification text may return `_en` / `_ar` fields or localized choice labels when `Accept-Language: ar` is set.

- **Face service messages:** Localized via `Accept-Language` header.

TODO: Document which dashboard labels are still English-only in the React UI (most sidebar labels in `navigation.js` are English strings).

8. Troubleshooting

9. Support Contacts

TODO: Insert school IT support email/phone for production deployment. *(Not in codebase.)*

Appendix H — Full Requirements (SRS Excerpt)

Software Requirements Specification (SRS)

> Derived from Django models, DRF viewsets, React routes, and microservice code.

- > **Version:** 1.0 (codebase audit, June 2026)

1. Introduction

1.1 Scope

The Smart School system provides a web application for managing school operations with four user roles, REST APIs, a React dashboard, a face-recognition attendance microservice, and an AI chatbot.

1.2 Definitions

2. Use Case Diagram Descriptions

2.1 Administrator

Actor: Administrator (`User.role = ADMIN`)

2.2 Teacher

Actor: Teacher (`User.role = TEACHER`)

2.3 Student

Actor: Student (`User.role = STUDENT`)

2.4 Parent

Actor: Parent (`User.role = PARENT`)

2.5 System (Face Recognition Service)

Actor: Face Recognition Service (automated, invoked by Django)

3. Functional Requirements by Feature

3.1 Users & Authentication

Business rules: Only ADMIN creates/updates/deletes users (`UserViewSet.get_permissions`). Students/parents see limited user lists (`get_queryset`). **APIs:** `POST /api/auth/login/`, `POST /api/auth/refresh/`, `/api/users/`, `/api/users/me/`

- **Tables:** `users`

Screens: `LoginPage.jsx`, `ProfilePage.jsx`, `AdminUsersPage.jsx`

3.2 Students

APIs: CRUD `/api/students/`, `POST .../register-face/`, `GET .../face-status/`, `POST .../backfill-ids/` (admin), `GET .../weekly-report/`, `GET .../dashboard/` (student)

- **Tables:** `students`, `students_subjects` (M2M)

Screens: `AdminStudentsPage.jsx`, `TeacherStudentsPage.jsx`, `StudentDashboard.jsx`

3.3 Teachers

APIs: `/api/teachers/`, dashboard action

- **Tables:** `teachers`, `teacher_subject_classes`, M2M tables

Screens: `AdminTeachersPage.jsx`, `TeacherDashboard.jsx`

3.4 Parents

APIs: `/api/parents/`, dashboard

- **Tables:** `parents`, `students.parent_id`

Screens: `AdminParentsPage.jsx`, `ParentDashboard.jsx`, `ParentChildrenPage.jsx`

3.5 Classes & Subjects

APIs: `/api/classes/`, `/api/subjects/`, `/api/materials/`

- **Tables:** `school_classes`, `subjects`, `materials`

Screens: `AdminClassesPage.jsx`, `AdminSubjectsPage.jsx`, `TeacherSubjectsPage.jsx`, `TeacherMaterialsPage.jsx`

3.6 Attendance (Manual + Face Recognition)

APIs: `/api/attendance/` (CRUD; teacher write; upsert on create) `/api/attendance/process-classroom-image/` (POST multipart) `/api/attendance-sessions/` (CRUD teacher; admin read) `GET .../active/`, `GET .../roster/`, `POST .../complete/`, `POST .../cancel/` `GET .../history/`, `GET .../class-history/?school_class=`

- **Tables:** `attendance`, `attendance_sessions`

Screens: `FeatureModulePage` (teacher attendance + camera), `SessionHistoryPage.jsx`, `StudentAttendancePage.jsx`, `ParentAttendancePage.jsx` **Sequence (face attendance):** See [system-architecture.md](./system-architecture.md#sequence-face-recognition-attendance)

3.7 Exams & Grades

APIs: `/api/exams/`, `/api/questions/`, `/api/grades/`, `GET /api/exams/upcoming/` (student)

- **Tables:** `exams`, `questions`, `grades`

Screens: `AdminExamsPage.jsx`, `TeacherExamsPage.jsx`, `StudentGradesPage.jsx`, `ParentGradesPage.jsx`

3.8 Weekly Reports

APIs: `/api/weekly-reports/`, `GET .../dashboard/`, `POST .../generate/`, `GET .../{id}/download-pdf/`

- **Tables:** `weekly_reports`

Screens: `AdminWeeklyReportsPage.jsx`, `TeacherWeeklyReportsPage.jsx`, `StudentWeeklyReportsPage.jsx` (API), `ParentWeeklyReportsPage.jsx` (client-side aggregate)

3.9 Videos

APIs: `/api/videos/`, `/api/video-progress/`, `POST .../sync/`

- **Tables:** `educational_videos`, `video_progress`

Screens: `TeacherVideosPage.jsx`, `StudentVideosPage.jsx`

3.10 Notifications

APIs: `/api/notifications/`, `POST .../mark-read/`, `POST .../mark-all-read/`, `/api/notification-preferences/`

- **Tables:** `notifications`, `notification_preferences`

Screens: `NotificationsPage.jsx`

3.11 AI Chatbot

APIs: `POST|GET /api/chatbot/ask/`

- **Tables:** Read-only across attendance, grades, students, etc.

Screens: `SmartChatbot.jsx`, `ParentChatbot.jsx`

4. Non-Functional Requirements (Observed)

TODO: Define formal SLA, max concurrent users, and backup RPO/RTO for production thesis deployment. *(Not in repository.)*

5. Dedicated System Chapters (Thesis-Ready Summaries)

5.1 Face Recognition Attendance

Purpose: Automate marking present students from a classroom photo during an active session.

- **Flow:**

1. Teacher creates `AttendanceSession` for a `SchoolClass` → absent records for all enrolled students. 2. Teacher uploads classroom image → Django calls FastAPI `detect-faces-batch` with class `student_ids`. 3. Matches update/create `Attendance` as `present`, `source=face_recognition`. 4. Teacher completes session → parents of still-absent students receive `ATTENDANCE` notifications.

- **Actors:** Teacher, Face Recognition Service, Parent (notified)

Tolerance: 0.6 default; HOG model default in client.

5.2 AI Chatbot Assistant

Purpose: Natural-language access to role-appropriate school data. **Mechanism:** Keyword intent → SQL-backed context strings → Gemini `generate_content` (max 300 tokens, temperature 0.3) or formatted fallback. **Constraints:** 500 character message limit; answers must use provided context only (prompt rules).

5.3 Localization (English / Arabic)

Backend: `APILanguageMiddleware`, `smartSchool/messages.py`, `locale/ar/`, modeltranslation `_en`/`_ar` columns. **Frontend:** i18next for login strings; `Accept-Language` from `localStorage` key `ss_lang`; RTL styling **TODO:** verify full-app RTL CSS coverage beyond login. **Face service:** Dictionary translations in `face_recognition_service/translations.py`.

5.4 Role-Based Access Control

Layers: 1. DRF permission classes (`IsAdmin`, `IsTeacher`, `IsAdminOrTeacher`, etc.) 2. Per-viewset `get_permissions()` action overrides 3. `get_queryset()` row-level filtering by role 4. React `RequireRole` route wrapper Utility helpers in `users/permissions.py`: `can_access_student_data`, `can_modify_student_data`.

6. Traceability Matrix (Sample)

Appendix I — Database Catalog Excerpt

Database Design

> ERD and table reference from Django models (`db_table` names).

- > Database engine: **Microsoft SQL Server** via `mssql-django`.

1. Entity-Relationship Description

1.1 Core identity

users (1) ←→ (0..1) **students** | **teachers** | **parents** via OneToOne `user_id` **users** (1) ←→ (0..*) **notifications** as `recipient_id` **users** (1) ←→ (0..1) **notification_preferences**

1.2 Academic structure

school_classes (1) ←→ (0..*) **students** (`school_class_id`) **school_classes** (M) ←→ (M) **teachers** via `teachers_assigned_classes` **subjects** (M) ←→ (M) **students** via `students_subjects` **subjects** (M) ←→ (M) **teachers** via `teachers_assigned_subjects` **teachers** + **subjects** + `class_id` string → **teacher_subject_classes** (junction with uniqueness)

1.3 Attendance

students (1) ←→ (0..*) **attendance** — unique (`student_id`, `date`) **attendance_sessions** (1) ←→ (0..*) **attendance** optional `session_id` **teachers** (1) ←→ (0..*) **attendance_sessions** as `instructor_id` **school_classes** (1) ←→ (0..*) **attendance_sessions**

1.4 Assessment

subjects (1) ←→ (0..*) **exams** **teachers** (1) ←→ (0..*) **exams** **exams** (1) ←→ (0..*) **questions** **students** + **exams** → **grades** — unique (`student_id`, `exam_id`)

1.5 Content & reports

subjects (1) ←→ (0..*) **materials**, **educational_videos** **teachers** upload materials/videos **students** + **educational_videos** → **video_progress** (unique pair) **students** (1) ←→ (0..*) **reports** **weekly_reports** optional FK **teachers** when `scope=TEACHER`

1.6 Face encodings (outside RDBMS)

Pickle files: `face_recognition_service/face_encodings/{student_id}.pkl` Linked logically by `students.student_id` string

2. ERD (Mermaid)

3. Table Catalog

3.1 users

Indexes: `role`

3.2 students

M2M: `subjects` → subjects

3.3 teachers

3.4 teacher_subject_classes

3.5 parents

3.6 school_classes

3.7 subjects

3.8 materials

3.9 attendance

Constraint: unique (`student`, `date`)

3.10 attendance_sessions

3.11 exams

3.12 questions

3.13 grades

3.14 reports

3.15 weekly_reports

3.16 notifications

3.17 notification_preferences

3.18 educational_videos

3.19 video_progress

4. Referential Integrity Summary

5. Indexes (Declared in Meta)

Notable indexes: `attendance (student, date)`, `notifications (recipient, -created_at)`, `weekly_reports (week_start, scope)`, exam/subject/teacher FK indexes.

6. Migration & Translation Notes

Bilingual columns created by `django-modeltranslation` migrations (e.g. `subjects/migrations/0005_*`, `notifications/migrations/0002_*`). Run `makemigrations` / `migrate` after changing `translation.py` files.

7. TODO

Production backup schedule and retention policy for SQL Server and `media/` uploads. *(Not in codebase.)* **Full list of M2M junction table names** as generated by Django on SQL Server — run `python manage.py sqlmigrate` in deployment environment if exact names required for DBA scripts.

Appendix J — API Reference Excerpt

API Documentation

> Base URL (development): `http://127.0.0.1:8000/api/`

- > Authentication: `Authorization: Bearer <access_token>` unless noted.

> Language: `Accept-Language: en|ar` or `?lang=ar`

- > Pagination: `PageNumberPagination`, `page_size=20` default.

1. Authentication

Login notes: `username` may be a student's `student_id` if no matching username exists. `user` includes `role`, `role_display`, optional `student_id`, `photo_url`.

2. Users

3. Students

4. Teachers

5. Parents

6. Classes

7. Subjects & Materials

8. Attendance

8.1 Attendance records

process-classroom-image response (success):

8.2 Attendance sessions

9. Exams, Questions, Grades

10. Reports

10.1 Student reports

10.2 Weekly reports

Generate body example:

11. Videos

12. Notifications

WebSocket: `ws://host/ws/notifications/` — JWT in connection (see `notifications/middleware.py`).

13. Chatbot

POST response:

14. Face Recognition Microservice

Base URL: `FACE_RECOGNITION_SERVICE_URL` (default `http://localhost:8001`) Called by Django only (not directly by React in production flow).

15. Root & Admin

16. Error Handling

DRF standard: `{ "detail": "..." }` or field errors. Localized messages from `smartSchool.messages` when middleware language is `ar`. Face/attendance endpoints often return `{ "success": false, "message", "error" }` with 4xx/5xx.

17. Standard ViewSet Operations

For each router-registered resource, DRF provides unless overridden: `GET /resource/` — list `POST /resource/` — create `GET /resource/{id}/` — retrieve `PUT/PATCH /resource/{id}/` — update `DELETE /resource/{id}/` — destroy Permissions vary per viewset; see [srs.md](./srs.md).

18. TODO

OpenAPI/Swagger schema generation is **not** present in the repo. **TODO:** Add `drf-spectacular` or export Postman collection if required for thesis appendix.